

NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero	NaveEspacial(o: char, c: entero) si c > maxDeposito combustible = maxDeposito, en caso contrario asigna c a combustible. Requiere el valor de o igual a 'V', 'R' o 'A'
<<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char)	NaveEspacial(o: char) combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A'
<<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito()	establecerColor(o: char) Requiere el valor de o igual a 'V', 'R' o 'A'
copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial	establecerCombustible(c: entero) si c > maxDeposito combustible = maxDeposito
	llenarDeposito() combustible = maxDeposito
	obtenerAutonomia(): real combustible/parsec

```
NaveEspacial
<<Atributos de clase>>
maxDeposito=1000 parsec=100
<<Atributos de instancia>>
color: char
combustible: entero
<<Constructores>>
NaveEspacial(o: char, c: entero)
NaveEspacial(o: char)
<<Comandos>>
establecerColor(o: char)
establecerCombustible(c: entero)
llenarDeposito()
copy(n:NaveEspacial)
<<Consultas>>
obtenerColor(): char
obtenerCombustible(): entero
obtenerAutonomia(): real
equals(n:NaveEspacial): boolean
clone(): NaveEspacial
```

copy(n:NaveEspacial) Si n está ligado, copia el estado interno del objeto ligado a la variable n, en el objeto que recibe el mensaje, en caso contrario no tiene ningún efecto
equals(n:NaveEspacial): boolean Requiere n ligado. Retorna verdadero si el estado interno del objeto que recibe el mensaje es igual al estado interno del objeto ligado a n
clone(): NaveEspacial Crea y retorna un nuevo objeto con el mismo estado interno que el objeto que recibe el mensaje

NaveEspacial	
<pre> <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero </pre>	<pre> class NaveEspacial { //Atributos de clase private static final int maxDeposito = 1000; private static final int parsec = 100; //Atributos de instancia private char color; private int combustible; </pre>
<pre> <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial </pre>	

La clase NaveEspacial

<pre> NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial </pre>	<pre> //Constructores public NaveEspacial (char o){ /*combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; combustible = maxDeposito; } </pre> Es responsabilidad de la clase cliente garantizar que el primer parámetro tenga valor 'V', 'R' o 'A'
--	--

La clase NaveEspacial

<pre> NaveEspacial <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(o: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial </pre>	<pre> //Constructores public NaveEspacial (char o){ /*combustible = maxDeposito Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; combustible = maxDeposito; } //Constructores public NaveEspacial (char o, int c){ /*si c > maxDeposito combustible = maxDeposito, en caso contrario asigna c a combustible. Requiere el valor de o igual a 'V', 'R' o 'A'*/ color = o; if (c > maxDeposito) combustible = maxDeposito; else combustible = c; } </pre>
--	--

Igualdad y equivalencia

```

NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
        
```



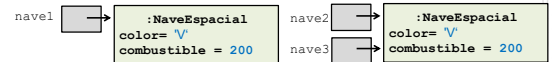
Las variables nave1 y nave2 mantienen distintas referencias pero están ligados a objetos **equivalentes**.

9

Igualdad y equivalencia

```

NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
        
```



Los valores de las variables nave2 y nave3 son iguales.

10

Igualdad y equivalencia

```

NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave1 == nave2)
        
```



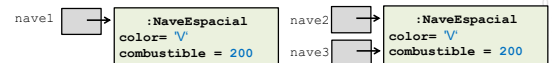
La expresión computa false porque nave1 y nave2 mantienen distintas referencias

11

Igualdad y equivalencia

```

NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('V',200);
NaveEspacial nave3 = nave2;
if (nave3 == nave2)
        
```



El operador == computa true porque nave3 y nave2 mantienen referencias a una misma nave

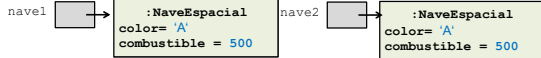
12

La consulta equals

```
public boolean equals (NaveEspacial n){
    /*Requiere n ligado. Retorna verdadero si el estado interno
    del objeto que recibe el mensaje es igual al estado interno
    del objeto ligado a n*/
    return color == n.obtenerColor() &&
           combustible == n.obteneCombustible();
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = new NaveEspacial ('A',500);
nave1.copy(nave2);
nave1.equals(nave2);
```

true



La consulta clone

```
public NaveEspacial clone (){
    /*Crea y retorna un nuevo objeto con el mismo estado interno
    que el objeto que recibe el mensaje*/
    return new NaveEspacial(color, combustible);
}
```

La consulta clone retorna una nueva nave con el mismo estado interno que la nave que recibió el mensaje.

La consulta clone

```
public NaveEspacial clone (){
    /*Crea y retorna un nuevo objeto con el mismo estado interno
    que el objeto que recibe el mensaje*/
    return new NaveEspacial(color, combustible);
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = nave1.clone();
```



La consulta clone

```
public NaveEspacial clone (){
    /*Crea y retorna un nuevo objeto con el mismo estado interno
    que el objeto que recibe el mensaje*/
    return new NaveEspacial(color, combustible);
}
```

```
NaveEspacial nave1 = new NaveEspacial ('V',200);
NaveEspacial nave2 = nave1.clone();
nave2.establecerColor('A');
```



Videojuego Aliens

En un Videojuego algunos de los personajes son **aliens**.

Los aliens tienen cierta cantidad de **antenas**, que determinan su capacidad sensora.

La cantidad de vidas se van reduciendo en 1 cada vez que recibe una herida. Cuando está muerto ya no tienen efecto las heridas. Cuando un alien logra llegar a la base recupera 2 vidas, sin superar nunca el valor 5.

Cada alien **puede** tener una nave, cada nave tiene un color y una cantidad de combustible en el tanque.

Los atributos vidas y antenas pueden aumentar o disminuir de acuerdo a un parámetro que puede ser positivo o negativo.

Si un alien está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicada por el número de vidas y luego se suma un quinto de la autonomía de la nave. Si no está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicada por el número de vidas.

Videojuego Aliens

```

NaveEspacial
<<Atributos de clase>>
maxDeposito=1000 parsec=100
<<Atributos de instancia>>
color: char
combustible: entero
<<Constructores>>
NaveEspacial(o: char, c: entero)
NaveEspacial(o: char)
<<Comandos>>
establecerColor(o: char)
establecerCombustible(c: entero)
llenarDeposito()
copy(n:NaveEspacial)
<<Consultas>>
obtenerColor(): char
obtenerCombustible(): entero
obtenerAutonomia(): real
equals(n:NaveEspacial): boolean
clone(): NaveEspacial
  
```

```

Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas: entero
antenas: entero
<<Constructor>>
Alien (n:NaveEspacial, v:a: entero)
<<Comandos>>
recuperaVidas()
recibeHerida()
copy(a: Alien)
establecerNave(n:NaveEspacial)
establecerAntenas(p: entero)
establecerVidas(p: entero)
  
```

Videojuego Aliens

NaveEspacial
<pre> <<Atributos de clase>> maxDeposito=1000 parsec=100 <<Atributos de instancia>> color: char combustible: entero <<Constructores>> NaveEspacial(o: char, c: entero) NaveEspacial(o: char) <<Comandos>> establecerColor(c: char) establecerCombustible(c: entero) llenarDeposito() copy(n:NaveEspacial) <<Consultas>> obtenerColor(): char obtenerCombustible(): entero obtenerAutonomia(): real equals(n:NaveEspacial):boolean clone():NaveEspacial </pre>

Alien
<pre> <<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero <<Consultas>> obtenerNave():NaveEspacial obtenerVidas():entero obtenerAntenas():entero obtenerFuerza():real clone():Alien equals(a: Alien): boolean </pre>

Videojuego Aliens

Las clases **Alien** y **NaveEspacial** están asociadas.

La clase **Alien** tiene un atributo de clase **NaveEspacial**.

La clase **Alien** es cliente de la clase proveedora **NaveEspacial**.

La clase **Alien** solo necesita conocer la **signatura** de los servicios provistos por **NaveEspacial**.

La clase **NaveEspacial** no necesita conocer a la clase **Alien** ni a ninguna de sus clases clientes.

Videojuego Aliens

La cantidad de vidas se van reduciendo en 1 cada vez que recibe una herida. Cuando está muerto ya no tienen efecto las heridas. Cuando un alien logra llegar a la base recupera 2 vidas, sin superar nunca el valor 5.

Alien(n:NaveEspacial, v:a:entero)
Requiere v <= 5 y a > 0.

copy(a:Alien)
Requiere a ligado, se implementa superficial

En este ejemplo parte de la funcionalidad de los servicios está especificada en el enunciado que precede al diagrama.

Alien
<pre> <<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero <<Constructor>> Alien (n:NaveEspacial,v:a:entero) <<Comandos>> recuperaVidas() recibeHerida() copy(a:Alien) establecerNave(n:NaveEspacial) establecerAntenas(p:entero) establecerVidas(p:entero) </pre>

Videojuego Aliens

Si un alien está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicada por el número de vidas y luego se suma un quinto de la autonomía de la nave. Si no está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicada por el número de vidas.

equals(a: Alien): boolean
Requiere a ligado, se implementa superficial

clone():Alien
Se implementa superficial

En este ejemplo parte de la funcionalidad de los servicios está especificada en el enunciado que precede al diagrama.

Alien
<pre> <<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero <<Consultas>> obtenerNave():NaveEspacial obtenerVidas():entero obtenerAntenas():entero obtenerFuerza():real clone():Alien equals(a: Alien): boolean </pre>

La clase Alien

obtenerFuerza():real
Si el alien está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicado por el número de vidas y luego se suma un quinto de la autonomía de la nave. Si no está asociado a una nave, su fuerza se calcula como la capacidad sensora, más su capacidad de lucha, todo multiplicado por el número de vidas

Los comentarios son otra forma de especificar las funcionalidades y responsabilidades

Alien
<pre> <<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero <<Consultas>> obtenerNave():NaveEspacial obtenerVidas():entero obtenerAntenas():entero obtenerFuerza():real clone():Alien equals(a: Alien): boolean </pre>

La clase Alien

- obtenerFuerza():real**
Si el alien está asociado a una nave, su fuerza se calcula como la capacidad sensora multiplicado por el número de vidas y luego se suma un quinto de la autonomía de la nave. Si no está asociado a una nave, su fuerza se calcula como la capacidad sensora, más su capacidad de lucha, todo multiplicado por el número de vidas.
- equals(a: Alien): boolean**
Requiere a ligado, se implementa superficial
- clone():Alien**
Se implementa superficial

En IPOO generalmente definimos la funcionalidad y las responsabilidades acompañando el diagrama con una especificación de cada servicio.

Alien
<pre> <<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero <<Consultas>> obtenerNave():NaveEspacial obtenerVidas():entero obtenerAntenas():entero obtenerFuerza():real clone():Alien equals(a: Alien): boolean </pre>

Videojuego Aliens

```
class Alien {
//Atributos de clase
private static final int maxVidas=5;
//Atributos de instancia
private NaveEspacial nave;
private int vidas;
private int antenas;
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Constructor>>
Alien (n:NaveEspacial,v:a:entero)
<<Comandos>>
recuperaVidas()
recibeHerida()
copy(a:Alien)
establecerNave(n:NaveEspacial)
establecerAntenas(p:entero)
establecerVidas(p:entero)
```

Videojuego Aliens

```
//Constructor
public Alien (NaveEspacial n,
int v, int a){
// Requiere v <=5
nave = n;
vidas = v;
antenas = a;
}
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Constructor>>
Alien (n:NaveEspacial,v:a:entero)
<<Comandos>>
recuperaVidas()
recibeHerida()
copy(a:Alien)
establecerNave(n:NaveEspacial)
establecerAntenas(p:entero)
establecerVidas(p:entero)
```

Cuando se crea un objeto de clase **Alien** puede quedar asociado a un objeto de clase **NaveEspacial**.

Videojuego Aliens

```
//Comandos
public void recuperaVidas(){
/*Cuando llega a la base recupera
dos vidas, sin superar las 5*/
if (vidas+2 > maxVidas)
vidas = maxVidas;
else
vidas = vidas+2; }

public void recibeHerida() {
/*Cuando recibe una herida pierde
una vida*/
if (vidas > 0) vidas--;
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Constructor>>
Alien (n:NaveEspacial,v:a:entero)
<<Comandos>>
recuperaVidas()
recibeHerida()
copy(a:Alien)
establecerNave(n:NaveEspacial)
establecerAntenas(p:entero)
establecerVidas(p:entero)
```

Videojuego Aliens

```
//Consultas
public NaveEspacial obtenerNave() {
return nave;
}
public int obtenerVidas() {
return vidas;
}
public int obtenerAntenas() {
return antenas;
}
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Consultas>>
obtenerNave():NaveEspacial
obtenerVidas():entero
obtenerAntenas():entero
obtenerFuerza():real
clone():Alien
equals(a: Alien): boolean
```

Videojuego Aliens

```
public float obtenerFuerza() {
/*Si el alien tiene nave, su
fuerza se calcula como la
capacidad sensora multiplicado por
el número de vidas y luego se suma
un quinto de la autonomia de la
nave. Si no tiene nave, su fuerza
se calcula como la capacidad
sensora, más su capacidad de
lucha, todo multiplicado por el
número de vidas.*/
float f=(float) antenas*vidas;
if (nave != null)
f=f+nave.obtenerAutonomia()/5;
return f; }
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Consultas>>
obtenerNave():NaveEspacial
obtenerVidas():entero
obtenerAntenas():entero
obtenerFuerza():real
clone():Alien
equals(a: Alien): boolean
```

Videojuego Aliens

```
public float obtenerFuerza() {
/*Si el alien tiene nave, su
fuerza se calcula como la
capacidad sensora multiplicado por
el número de vidas y luego se suma
un quinto de la autonomia de la
nave. Si no tiene nave, su fuerza
se calcula como la capacidad
sensora, más su capacidad de
lucha, todo multiplicado por el
número de vidas.*/
float f=(float) antenas*vidas;
if (nave != null)
f=f+nave.obtenerAutonomia()/5;
return f; }
```

```
Alien
<<Atributos de clase>>
maxVidas=5
<<Atributos de instancia>>
nave: NaveEspacial
vidas:entero
antenas:entero

<<Consultas>>
obtenerNave():NaveEspacial
obtenerVidas():entero
obtenerAntenas():entero
obtenerFuerza():real
clone():Alien
equals(a: Alien): boolean
```

La consulta **obtenerFuerza** de la clase **Alien** envía el mensaje **obtenerAutonomia** a un objeto de la clase asociada **NaveEspacial**.

Videojuego Aliens

```
public float obtenerFuerza() {
    /*Si el alien tiene nave, su
    fuerza se calcula como la
    capacidad sensora multiplicado por
    el número de vidas y luego se suma
    un quinto de la autonomia de la
    nave. Si no tiene nave, su fuerza
    se calcula como la capacidad


Antes de enviar el mensaje se controla
    que el alien esté asociado a una nave.



```
float f=(float) antenas*vidas;
if (nave != null)
 f=f+nave.obtenerAutonomia()/5;
return f; }
```



| Alien                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>&lt;&lt;Atributos de clase&gt;&gt;</b><br>maxVidas=5<br><b>&lt;&lt;Atributos de instancia&gt;&gt;</b><br>nave: NaveEspacial<br>vidas:entero<br>antenas:entero                         |
| <b>&lt;&lt;Consultas&gt;&gt;</b><br>obtenerNave():NaveEspacial<br>obtenerVidas():entero<br>obtenerAntenas():entero<br>obtenerFuerza():real<br>clone():Alien<br>equals(a: Alien): boolean |


```

Videojuego Aliens

```
public void copy (Alien a){
    /*Requiere a ligada, implementa
    copy superficial*/
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas= a.obtenerAntenas();
}
```

Alien
<<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero
<<Constructor>> Alien (n:NaveEspacial,v,a:entero) <<Comandos>> recuperaVidas() recibeHerida() copy(a:Alien) establecerNave(n:NaveEspacial) establecerAntenas(p:entero) establecerVidas(p:entero)

El comando **copy** asigna al alien que recibe el mensaje la misma cantidad de vidas y antenas que el alien que pasa como parámetro y lo asocia también a la misma nave.

Videojuego Aliens

```
public void copy (Alien a){
    /*Requiere a ligada, implementa
    copy superficial*/
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas= a.obtenerAntenas();
}
```

Si en la clase Alien el comando **copy** se implementara en profundidad, debería controlar que la nave esté ligada, antes de enviarle el mensaje **copy** a dicha nave.

Alien
<<Atributos de clase>> maxVidas=5 <<Atributos de instancia>> nave: NaveEspacial vidas:entero antenas:entero
<<Constructor>> Alien (n:NaveEspacial,v,a:entero) <<Comandos>> recuperaVidas() recibeHerida() copy(a:Alien) establecerNave(n:NaveEspacial) establecerAntenas(p:entero) establecerVidas(p:entero)

Videojuego Aliens

```
public void copy (Alien a){
    /*Requiere a ligada, implementa
    copy superficial*/
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas= a.obtenerAntenas();
}
```

a1.copy(a2);

```
graph LR
    a1[Alien: nave, vidas=3, antenas=2] -- nave --> n1[NaveEspacial: color='V', combustible=550]
    a2[Alien: nave, vidas=5, antenas=4] -- nave --> n2[NaveEspacial: color='V', combustible=550]
```

Videojuego Aliens

```
public void copy (Alien a){
    /*Requiere a ligada, implementa copy superficial*/
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas= a.obtenerAntenas();
}
```

a1.copy(a2);

```
graph LR
    a1[Alien: nave, vidas=3, antenas=2] -- nave --> n1[NaveEspacial: color='V', combustible=550]
    a2[Alien: nave, vidas=5, antenas=4] -- nave --> n2[NaveEspacial: color='V', combustible=550]
```

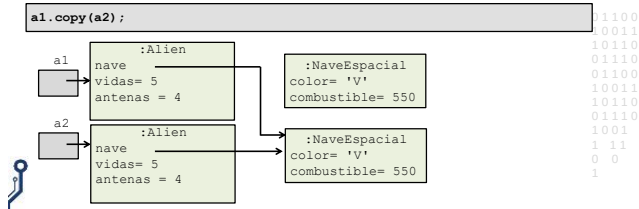
Videojuego Aliens

```
public void copy (Alien a){
    /*Requiere a ligada, implementa copy superficial*/
    nave = a.obtenerNave();
    vidas = a.obtenerVidas();
    antenas= a.obtenerAntenas();
}
```

a1.copy(a2);

```
graph LR
    a1[Alien: nave, vidas=5, antenas=4] -- nave --> n1[NaveEspacial: color='V', combustible=550]
    a2[Alien: nave, vidas=5, antenas=4] -- nave --> n2[NaveEspacial: color='V', combustible=550]
```

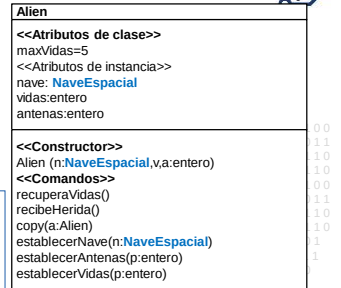
Videojuego Aliens



Videojuego Aliens

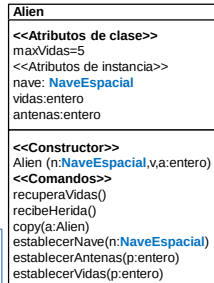
```
public Alien clone(){
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}
```

La consulta `clone` crea y devuelve un alien con la misma cantidad de vidas y antenas que el alien que recibe el mensaje y asociado también a la misma nave.



Videojuego Aliens

```
public Alien clone(){
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}
```

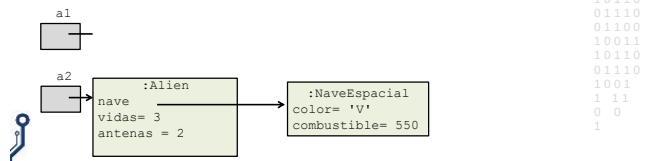


Si el alien que recibe el mensaje clone no está asociado a una nave, el nuevo alien tampoco lo estará.

Videojuego Aliens

```
public Alien clone(){
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}
```

```
a1 = a2.clone();
```



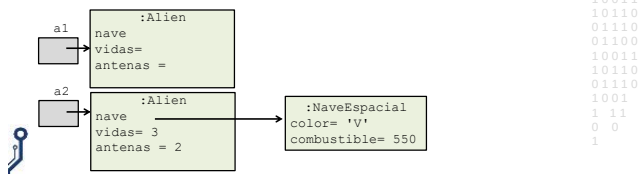
Videojuego Aliens

```
public Alien clone(){
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}
```

```

//Constructor
public Alien (NaveEspacial n,
    int v, int a){
    // Requiere v <=5
    nave = n;
    vidas = v;
    antenas = a;
}
```

```
a1 = a2.clone();
```



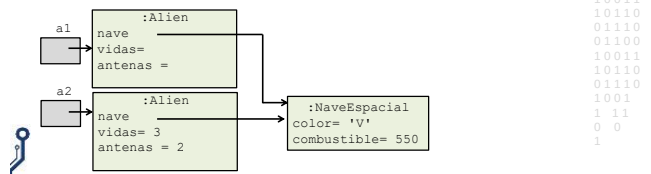
Videojuego Aliens

```
public Alien clone(){
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}
```

```

//Constructor
public Alien (NaveEspacial n,
    int v, int a) {
    // Requiere v <=5
    nave = n;
    vidas = v;
    antenas = a;
}
```

```
a1 = a2.clone();
```

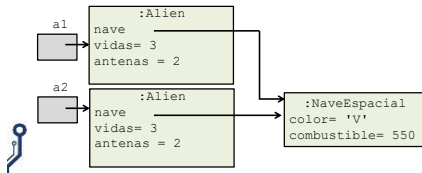


Videojuego Aliens

```
public Alien clone() {
    //Se implementa superficial
    return new Alien(nave,vidas,
        antenas);
}

//Constructor
public Alien (NaveEspacial n,
    int v, int a){
    // Requiere v <=5
    nave = n;
    vidas = v;
    antenas = a;
}
```

```
a1 = a2.clone();
```



Videojuego Aliens

```
public boolean equals (Alien a){
    /*Requiere a ligada, se implementa superficial*/
    return (nave == a.obtenerNave() &&
        vidas == a.obtenerVidas() &&
        antenas == a.obtenerAntenas());
}
```

La consulta `equals` retorna true si el alien que recibe el mensaje y el que pasa como parámetro son equivalentes.

```

class Alien {
    <<Atributos de clase>>
    maxVidas=5
    <<Atributos de instancia>>
    nave: NaveEspacial
    vidas: entero
    antenas: entero

    <<Constructor>>
    Alien (n:NaveEspacial,v:a:entero)

    <<Comandos>>
    recuperaVidas()
    recibeHerida()
    copy(a:Alien)
    establecerNave(n:NaveEspacial)
    establecerAntenas(p:entero)
    establecerVidas(p:entero)
}
    
```

Videojuego Aliens

```
public boolean equals (Alien a){
    /*Requiere a ligada, se implementa superficial*/
    return (nave == a.obtenerNave() &&
        vidas == a.obtenerVidas() &&
        antenas == a.obtenerAntenas());
}
```

En este diseño, dos aliens son equivalentes si tienen la misma cantidad de vidas y antenas y ambos están asociados a la misma nave. Es decir los atributos de instancia `nave` de los dos objetos de clase `Alien` tienen la misma **identidad**.

```

class Alien {
    <<Atributos de clase>>
    maxVidas=5
    <<Atributos de instancia>>
    nave: NaveEspacial
    vidas: entero
    antenas: entero

    <<Constructor>>
    Alien (n:NaveEspacial,v:a:entero)

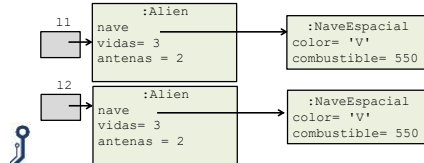
    <<Comandos>>
    recuperaVidas()
    recibeHerida()
    copy(a:Alien)
    establecerNave(n:NaveEspacial)
    establecerAntenas(p:entero)
    establecerVidas(p:entero)
}
    
```

Videojuego Aliens

```
public boolean equals (Alien a){
    /*Requiere a ligada, se implementa superficial*/
    return (nave == a.obtenerNave() &&
        vidas == a.obtenerVidas() &&
        antenas == a.obtenerAntenas());
}
```

```
11.equals(12);
```

false

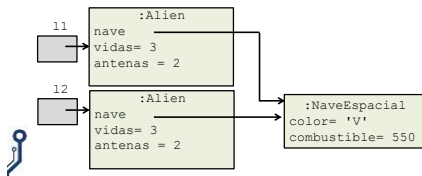


Videojuego Aliens

```
public boolean equals (Alien a){
    /*Requiere a ligada, se implementa superficial*/
    return (nave == a.obtenerNave() &&
        vidas == a.obtenerVidas() &&
        antenas == a.obtenerAntenas());
}
```

```
11.equals(12);
```

true



Videojuego Aliens

```

class VideojuegoAliens {
    public static void main (String s[]){
        NaveEspacial n1,n2,n3;
        Alien a1,a2,a3;
        n1 = new NaveEspacial ('V',550);
        n2 = new NaveEspacial ('V',550);
        n3 = n2;
        a1 = new Alien(n1,3,2);
        a2 = new Alien(n2,2,3);
        a3 = a1.clone();
        a4 = a1;
        System.out.println(a1==a2);
        System.out.println(a1==a3);
        System.out.println(a1==a4);
        a4.copy(a2);
        a1.recibeHerida();
        System.out.println(a1==a2);
        System.out.println(a1==a3);
        System.out.println(a1==a4);
    }
}
    
```

Videojuego Aliens

```
class VideojuegoAliens{
public static void main (String s[]){
    NaveEspacial n1,n2,n3;
    Alien a1,a2,a3;
    n1 = new NaveEspacial ('V',550);
    n2 = new NaveEspacial ('V',550);
    n3 = n2;
    a1 = new Alien(n1,3,2);
    a2 = new Alien(n2,2,3);
    a3 = a1.clone();
    a4 = a1;
    System.out.println(a1.equals((a2));
    System.out.println(a1.equals((a3));
    System.out.println(a1.equals((a4));
    a4.copy(a2);
    a1.recibeHerida();
    System.out.println(a1.equals((a2));
    System.out.println(a1.equals((a3));
    System.out.println(a1.equals((a4));
}
```

Las clases asociadas

En **copy superficial** se copia el estado interno del objeto referenciado por el parámetro en el estado interno del objeto que recibe el mensaje. Particularmente, para los atributos de tipo clase, se copia la **referencia** al objeto de la clase asociada.

En **clone superficial** se genera un clone con el mismo estado interno del objeto que recibe el mensaje. Particularmente, en los atributos de tipo clase, se almacena la **referencia** al mismo objeto de la clase asociada.

En la **equals superficial** se compara el estado interno de los objetos. Particularmente, los atributos de instancia de tipo clase se compara la **referencia** al objeto de la clase asociada.

Videojuego Androides

En un Videojuego algunos de los personajes son **Androides**.

Los androides pueden aprender a hablar, a medida que el juego avanza pueden adquirir nuevos vocablos.

Cada androide **tiene una nave**, cada nave tiene un **color** y una **cantidad de combustible** en el tanque.

Ambos atributos pueden aumentar o disminuir de acuerdo a un parámetro que puede ser positivo o negativo.

Videojuego Androide

```
NaveEspacial
<<Atributos de clase>>
maxDeposito=1000 parsec=100
<<Atributos de instancia>>
color: char
combustible: entero
<<Constructores>>
NaveEspacial(o: char, c: entero)
NaveEspacial(o: char)
<<Comandos>>
establecerColor(o: char)
establecerCombustible(c: entero)
llenarDeposito()
copy(n:NaveEspacial)
<<Consultas>>
obtenerColor(): char
obtenerCombustible(): entero
obtenerAutonomia(): real
equals(n:NaveEspacial): boolean
clone():NaveEspacial
```

```
Androide
<<Atributos de instancia>>
vocablos: entero
nave: NaveEspacial
<<Constructor>>
Androide (n: NaveEspacial)
<<Comandos>>
establecerNave(n: NaveEspacial)
aprenderVocablos(v: entero)
copy(a: Androide)
obtenerNave(): NaveEspacial
obtenerVocablos(): entero
clone(): Androide
equals(a: Androide): boolean
```

Videojuego Androide

Androide (n: NaveEspacial)
Requiere n ligada, inicializa vocablos en 0

establecerNave(n:NaveEspacial)
Si n está ligada asocia el androide a la nave n

aprenderVocablos(v: entero)
Requiere v mayor a 0. Aumenta vocablos en el valor de v

```
Androide
<<Atributos de instancia>>
vocablos: entero
nave: NaveEspacial
<<Constructor>>
Androide (n: NaveEspacial)
<<Comandos>>
establecerNave(n: NaveEspacial)
aprenderVocablos(v: entero)
copy(a: Androide)
<<Consultas>>
obtenerNave(): NaveEspacial
obtenerVocablos(): entero
clone(): Androide
equals(a: Androide): boolean
```

Todo androide está siempre asociado a una nave

Videojuego Androide

copy(a:Androide)
Requiere a ligado, se implementa en profundidad

equals(a: Androide): boolean
Requiere a ligado, se implementa en profundidad

clone(): Androide
Se implementa en profundidad

```
Androide
<<Atributos de instancia>>
vocablos: entero
nave: NaveEspacial
<<Constructor>>
Androide (n: NaveEspacial)
<<Comandos>>
establecerNave(n: NaveEspacial)
aprenderVocablos(v: entero)
copy(a: Androide)
<<Consultas>>
obtenerNave(): NaveEspacial
obtenerVocablos(): entero
clone(): Androide
equals(a: Androide): boolean
```

Videojuego Androide

```
class Androide {
    //Atributos de instancia
    private int vocablos;
    private NaveEspacial nave;
    //Constructor
    public Androide (NaveEspacial n) {
        // Requiere n ligado
        vocablos = 0; nave = n;}
}
```

Androide	
<<Atributos de instancia>>	vocablos: entero nave: NaveEspacial
<<Constructor>>	Androide (n: NaveEspacial)
<<Comandos>>	establecerNave(n: NaveEspacial) aprenderVocablos(v: entero) copy(a: Androide)
<<Consultas>>	obtenerNave(): NaveEspacial obtenerVocablos(): entero clone(): Androide equals(a: Androide): boolean

Todo androide se crea asociado a una nave

Videojuego Androide

```
public void
establecerNave(NaveEspacial n) {
    /*Si n está ligada asocia el
    androide a la nave n*/
    if (n != null)
        nave = n;
}

public void
aprenderVocablos (int v) {
    /*Requiere v mayor a 0,
    aumenta vocablos en v*/
    vocablos = vocablos + v;
}
```

Androide	
<<Atributos de instancia>>	vocablos: entero nave: NaveEspacial
<<Constructor>>	Androide (n: NaveEspacial)
<<Comandos>>	establecerNave(n: NaveEspacial) aprenderVocablos(v: entero) copy(a: Androide)
<<Consultas>>	obtenerNave(): NaveEspacial obtenerVocablos(): entero clone(): Androide equals(a: Androide): boolean

Videojuego Androide

```
public NaveEspacial
obtenerNave() {
    return nave;
}

public int obtenerVocablos() {
    return vocablos;
}
```

Androide	
<<Atributos de instancia>>	vocablos: entero nave: NaveEspacial
<<Constructor>>	Androide (n: NaveEspacial)
<<Comandos>>	establecerNave(n: NaveEspacial) aprenderVocablos(v: entero) copy(a: Androide)
<<Consultas>>	obtenerNave(): NaveEspacial obtenerVocablos(): entero clone(): Androide equals(a: Androide): boolean

Como el atributo de instancia **nave** siempre está ligado, el resultado siempre es una referencia a una nave.

Videojuego Androides

```
class VideojuegoAndroides{
    public static void main (String s[]){
        NaveEspacial n1,n2,n3;
        Androide a1,a2,a3;
    }
}
```

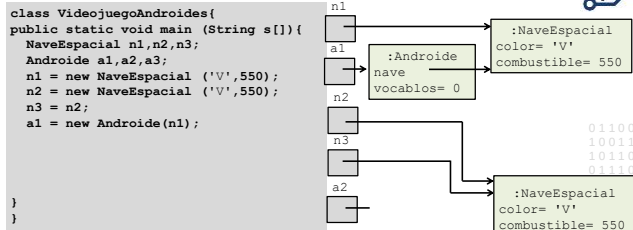
Videojuego Androides

```
class VideojuegoAndroides{
    public static void main (String s[]){
        NaveEspacial n1,n2,n3;
        Androide a1,a2,a3;
        n1 = new NaveEspacial ('V',550);
        n2 = new NaveEspacial ('V',550);
    }
}
```

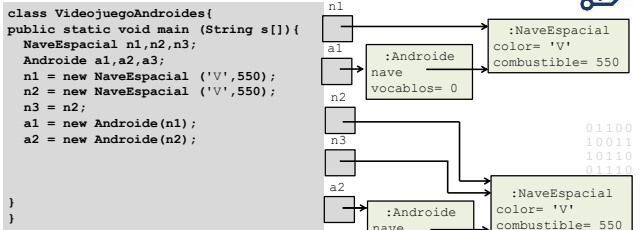
Videojuego Androides

```
class VideojuegoAndroides{
    public static void main (String s[]){
        NaveEspacial n1,n2,n3;
        Androide a1,a2,a3;
        n1 = new NaveEspacial ('V',550);
        n2 = new NaveEspacial ('V',550);
        n3 = n2;
    }
}
```

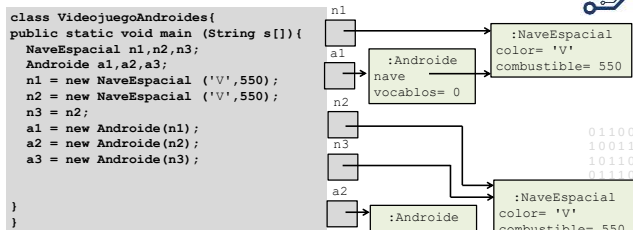
Videojuego Androides



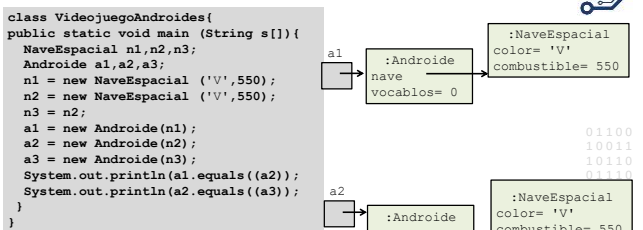
Videojuego Androides



Videojuego Androides



Videojuego Androides



Elimino algunas variables para concentrarnos en los androides

equals en profundidad

```
public boolean equals (Androide a){
    /*Requiere a ligado. Se implementa en profundidad*/
    return vocablos == a.obtenerVocablos() &&
        nave.equals(a.obtenerNave());
}
```

equals en profundidad retorna true si androide que recibe el mensaje conoce la misma cantidad de vocablos que el androide **a** y asociado a una nave con el mismo estado interno que la nave de **a**.

copy en profundidad

```
public void copy (Androide a){
    /*Requiere a ligado, se implementa en profundidad*/
    vocablos = a.obtenerVocablos();
    nave.copy(a.obtenerNave());
}
```

copy en profundidad, modifica el estado interno de la nave del androide que recibe el mensaje, que pasa a tener el mismo estado interno de la nave del androide **a**.

clone en profundidad

```
public Androide clone(){
    /*Se implementa en profundidad*/
    return new Androide(nave.clone());
}
```

Con esta implementación, el nuevo androide NO tiene vocabulario

clone en profundidad

```
public Androide clone(){
    /*Se implementa en profundidad*/
    Androide a = new Androide(nave.clone());
    a.aprenderVocablos(vocablos);
    return a;
}
```

Para generar un clon usamos el constructor y luego enviamos el mensaje `aprenderVocablos` al nuevo androide.

Las clases asociadas

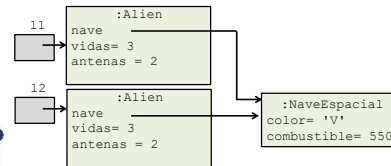
En **copy en profundidad**, a diferencia de la copia superficial, se copia el **estado interno** del objeto de la clase asociada.

En **clone en profundidad**, a diferencia del clone superficial, se genera un clone con un nuevo objeto de la clase asociada.

En la **equals en profundidad**, a diferencia del equals superficial, se compara el estado interno del objeto de la clase asociada.

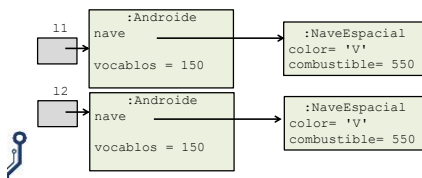
Las clases asociadas

Si `11.equals(12)` retorna true los objetos son **equivalentes**. Si `equals` se implementa en forma superficial la equivalencia es superficial, exige que los dos objetos tengan exactamente el mismo estado interno, es decir los mismos valores en cada uno de los atributos.



Las clases asociadas

Si `11.equals(12)` retorna true los objetos son **equivalentes**. Si `equals` se implementa en profundidad la equivalencia es profunda, exige que los dos objetos tengan los mismos valores en los atributos de tipo elemental y estén asociados a objetos equivalentes.



Las clases asociadas

Es el diseñador de un sistema, considerando las características específicas del problema, el que determina si la igualdad debe implementarse en forma superficial o en profundidad.

Es decir, si para que dos objetos sean equivalentes deben estar asociados a los mismos objetos o es suficiente con que estén asociados a objetos equivalentes.

El diseñador puede determinar además que la clase cliente es responsable de garantizar que el parámetro está ligado o bien puede especificar que la clase proveedora debe controlar que el parámetro no sea nulo. Si no especifica ni lo uno ni lo otro, como programadores debemos incluir los controles que eviten los errores de ejecución.

Relaciones entre clases

Asociación

La **asociación** es una forma de relación entre clases y se produce cuando el modelo de un objeto del problema **contiene o puede contener** al modelo de otro objeto del problema.

Dependencia

La **dependencia** es una forma de relación entre clases y se produce cuando el modelo de un objeto del problema **usa** al modelo de otro objeto.

1100
0011
0110
1110
1100
0011
0110
1110
001
11
0
1

Recomendaciones

Lean el material publicado

Asistir activamente a las clases es valioso para entender y relacionar los contenidos de IPOO, leer el material es también importante para reforzar la comprensión de estos contenidos antes de aplicarlos a la resolución de problemas.

01100
10011
10110
01110
01100
10011
10110
01110
1001
111
00
1

Recomendaciones

Practiquemos “definir” y “explicar”

```
boolean equi = o1.copy(o2)
```

- Defina el concepto de clases.
- Explique la relación de asociación entre clases.
- Si o1 y o2 son de clase A y la clase A está asociada a B, ¿cuál es la diferencia entre copy superficial y copy en profundidad? ¿cuál es más estricto?

01100
10011
10110
01110
01100
10011
10110
01110
1001
111
00
1